



INSTITUTO POLITÉCNICO NACIONAL
UNIDAD PROFESIONAL INTERDISCIPLINARIA DE INGENIERÍA
Y CIENCIAS SOCIALES Y ADMINISTRATIVAS



Licenciatura en Ciencias de la Informática

Tercer Departamental

Abstracción y Uso de Datos (3CM30)

Alumno:

Carrera Rojo Jane Adriana
Flores Maya Valeria
Frías Martínez Vanessa Judith
Martínez Valencia Karla Itzel
Saavedra Rodríguez Aarón Emanuel

Docente:

Entzana Garduño Yventz

18 de junio de 2026

Índice general

1	Introducción	2
2	La librería <code>output_writer.h</code>	2
2.1	Propósito	2
2.2	Arquitectura general	2
2.3	TeeBuffer: duplicar la salida	3
2.4	OutputCapture: el patrón RAII	3
2.5	Las cuatro funciones de escritura	4
2.5.1	JSON — <code>writeJSON</code>	4
2.5.2	XML — <code>writeXML</code>	4
2.5.3	CSV — <code>writeCSV</code>	4
2.5.4	TXT — <code>writeTXT</code>	4
2.5.5	<code>writeAllOutputs</code>	4
2.6	Cómo se usa en cada programa	5
2.7	Diferencia entre <code>1erDep</code> y <code>2doDep</code> : la ruta de salida	5
2.8	Ejemplo de salida generada	6
3	Programa <code>EstructurasCasaMetro</code>	6
3.1	Visión general	6
3.2	Organización de archivos	6
3.3	Estructuras de datos	7
3.4	La clase base <code>GrafoADT</code>	7
3.4.1	<code>cargar y agregarNodo</code>	8
3.4.2	El algoritmo de Dijkstra	8
3.5	Las tres variantes	9
3.5.1	<code>Grafo</code> (no dirigido)	9
3.5.2	<code>Digrafo</code> (dirigido)	9
3.5.3	<code>Arbol</code> (BFS desde una raíz)	9
3.6	El lector de archivos (<code>parser</code>)	9
3.7	El flujo principal (<code>main.cpp</code>)	10
3.8	Ejemplos de ejecución	10
3.9	Compilación y ejecución	11
4	Conclusiones	11

1 Introducción

Este documento describe la parte del proyecto **3erDep** relacionada con la **centralización de salidas** de los programas de los dos primeros departamentales y, por separado, el funcionamiento **completo** del programa **EstructurasCasaMetro**.

El proyecto se organiza en tres carpetas principales:

Carpeta	Contenido
1erDep_outs	Programas del primer departamental (HolaMundo, Calculadoras, recursividad, etc.) instrumentados para volcar su salida a un archivo común mediante <code>output_writer.h</code> .
2doDep_outs	Programas del segundo departamental (Pilas, Colas, Listas y algoritmos de ordenamiento) con la <i>misma</i> librería <code>output_writer.h</code> , adaptada a subdirectorios más profundos.
EstructurasCasaMetro	Programa autónomo que modela una red de transporte (Metro / Metrobús) como grafo , dígrafo y árbol , y calcula la ruta más corta con Dijkstra .

Alcance de este reporte. Para `1erDep_outs` y `2doDep_outs` *sólo* se documenta la librería `output_writer.h` y la forma en que los programas la usan para concentrar su salida en un único documento (`.json`, `.xml`, `.csv` o `.txt`). Para `EstructurasCasaMetro` se documenta el programa completo.

2 La librería `output_writer.h`

2.1 Propósito

Cada ejercicio de los dos primeros departamentales es un programa independiente que imprime sus resultados en la terminal con `std::cout`. El problema que resuelve `output_writer.h` es: *¿cómo guardar lo que cada programa imprime en un mismo conjunto de archivos, sin reescribir la lógica de cada ejercicio?*

La librería logra esto con dos ideas combinadas:

1. **Interceptar** todo lo que se manda a `std::cout` *sin* dejar de mostrarlo en pantalla (un “tee”).
2. **Volcar** ese texto capturado, al terminar el programa, a cuatro formatos a la vez: `out.json`, `out.xml`, `out.csv` y `out.txt`.

Es un *header-only* (todo está en el `.h`, con funciones `inline`): basta incluirlo, sin compilar archivos extra.

2.2 Arquitectura general

Componente	Responsabilidad
TeeBuffer	<code>streambuf</code> que duplica cada byte hacia dos destinos: la terminal y un buffer de captura.
OutputCapture	Objeto RAII que instala el <code>TeeBuffer</code> en <code>cout</code> al construirse y lo restaura al destruirse.
OUTPUT_DIR	Macro con la ruta (relativa) donde se escriben los cuatro archivos.
writeJSON/XML/CSV/TXT	Cuatro funciones que serializan el texto capturado a cada formato, escapando los caracteres especiales de cada uno.
writeAllOutputs	Función de conveniencia que llama a las cuatro anteriores.

2.3 TeeBuffer: duplicar la salida

TeeBuffer hereda de `std::streambuf` y redefine los dos puntos por donde pasa la escritura de un *stream*: `overflow` (un carácter a la vez) y `xspn` (un bloque de caracteres). En ambos casos, lo que recibe lo reenvía *tanto* a la terminal como al buffer de captura.

```

1  class TeeBuffer : public std::streambuf {
2  public:
3      TeeBuffer(std::streambuf* terminal, std::streambuf* capture)
4          : terminal_(terminal), capture_(capture) {}
5  protected:
6      int overflow(int c) override {           // un carácter
7          if (c == EOF) return EOF;
8          if (terminal_>sputc(c) == EOF) return EOF;
9          if (capture_>sputc(c) == EOF) return EOF;
10         return c;
11     }
12     std::streamsize xspn(const char* s, std::streamsize n) override {
13         terminal_>sputn(s, n);                // bloque de caracteres
14         capture_>sputn(s, n);
15         return n;
16     }
17 private:
18     std::streambuf* terminal_;
19     std::streambuf* capture_;
20 };
    
```

De esta forma, el usuario sigue viendo la salida normal en pantalla, y al mismo tiempo todo queda copiado en el `ostringstream` de captura.

2.4 OutputCapture: el patrón RAI

OutputCapture es la pieza que el programador usa directamente. En su **constructor** cambia el `streambuf` interno de `std::cout` por el TeeBuffer; en su **destructor** restaura el original. Como la restauración ocurre automáticamente al salir del *scope*, no hay riesgo de dejar a `cout` “roto” aunque el programa termine por una excepción.

```

1  class OutputCapture {
2  public:
3      OutputCapture() {
4          tee_ = new TeeBuffer(std::cout.rdbuf(), capture_.rdbuf());
5          original_ = std::cout.rdbuf(tee_);    // instala el tee en cout
6      }
7      ~OutputCapture() {
8          std::cout.rdbuf(original_);          // restaura cout
9          delete tee_;
10     }
11     std::string get() const { return capture_.str(); } // texto capturado
12 private:
13     std::ostringstream capture_;
14     TeeBuffer* tee_;
15     std::streambuf* original_;
16 };
    
```

RAII (*Resource Acquisition Is Initialization*): el recurso —en este caso, el “desvío” de `cout`— se adquiere en el constructor y se libera en el destructor. El programador no tiene que acordarse de “desinstalar” nada.

2.5 Las cuatro funciones de escritura

Cada función recibe el **nombre del programa** y el **texto capturado**, y produce un archivo con `std::ios::trunc` (sobrescribe la ejecución anterior). La diferencia entre ellas es el *formato* y, sobre todo, el **escape** de caracteres especiales propio de cada uno.

2.5.1 JSON — writeJSON

Escapa comillas, contrabarras y los saltos de línea / tabuladores (`\n`, `\r`, `\t`) para que el texto sea una cadena JSON válida.

```

1 inline void writeJSON(const std::string& programa, const std::string& salida) {
2     std::string path = std::string(OUTPUT_DIR) + "out.json";
3     std::string escaped;
4     for (char c : salida) {
5         if (c == '"') escaped += "\\\"";
6         else if (c == '\\') escaped += "\\\"";
7         else if (c == '\n') escaped += "\\n";
8         else if (c == '\r') escaped += "\\r";
9         else if (c == '\t') escaped += "\\t";
10        else          escaped += c;
11    }
12    std::ofstream f(path, std::ios::trunc);
13    f << "[\n  {\n"
14        << "    \"programa\": \"" << programa << "\",\n"
15        << "    \"salida\": \"" << escaped << "\",\n"
16        << "  }\n]\n";
17 }

```

2.5.2 XML — writeXML

Escapa las cinco entidades de XML (&, <, >, ", ') y envuelve el texto en una estructura `<programas><entrada>...`

2.5.3 CSV — writeCSV

Duplica las comillas (regla de *escape* de CSV) y encierra cada campo entre comillas, generando un encabezado `programa,salida` y una fila de datos.

2.5.4 TXT — writeTXT

Es el más simple: escribe el texto tal cual entre dos líneas separadoras y un encabezado legible. No requiere escape.

2.5.5 writeAllOutputs

Punto de entrada único: llama a las cuatro funciones anteriores con el mismo par (`programa, salida`).

```

1 inline void writeAllOutputs(const std::string& programa, const std::string& salida) {
2     writeJSON(programa, salida);
3     writeXML (programa, salida);
4     writeCSV (programa, salida);
5     writeTXT (programa, salida);
6 }

```

2.6 Cómo se usa en cada programa

El patrón de uso es idéntico en todos los ejercicios y consta de tres pasos: incluir el *header*, declarar un `OutputCapture` al inicio de `main`, y llamar a `writeAllOutputs` al final. Ejemplo real de `1erDep_outs/HolaMundo/main.cpp`:

```
1 #include <iostream>
2 #include "../output_writer.h"    // (1) incluir la librería
3 #include "holamundo.h"
4 #include "holamundo.cpp"
5 using namespace std;
6
7 int main(int argc, char** argv){
8     OutputCapture cap;           // (2) empieza a capturar cout
9
10    HolaMundo obj1 = HolaMundo();
11    obj1.saludar();               // todo cout se muestra Y se captura
12
13    writeAllOutputs("HolaMundo", cap.get()); // (3) vuelca a los 4 archivos
14    return 0;
15 }
```

El nombre que se pasa como primer argumento ("HolaMundo") es la *etiqueta* que identifica al programa dentro de los archivos de salida.

2.7 Diferencia entre 1erDep y 2doDep: la ruta de salida

Las dos versiones de la librería son casi idénticas. La diferencia clave está en la macro `OUTPUT_DIR`, que indica *dónde* se crean los cuatro archivos respecto al ejecutable.

En `1erDep_outs` los programas viven un nivel debajo de la carpeta de salida, así que la ruta es fija:

```
1 #define OUTPUT_DIR "../"
```

En `2doDep_outs` los programas están en subdirectorios de *profundidad variable* (por ejemplo `Pilas/Arreglos/DatoBase/`). Por eso la macro se define con guarda `#ifndef`, de modo que cada `main.cpp` pueda **redefinirla** antes de incluir el *header* según su profundidad:

```
1 // dentro de output_writer.h (2doDep): valor por omisión = profundidad 3
2 #ifndef OUTPUT_DIR
3 #define OUTPUT_DIR "../..."
4 #endif
```

```
1 // dentro de SortBubble/BubbleArr/main.cpp (profundidad 2): se redefine antes
2 #include <iostream>
3 #define OUTPUT_DIR "../..." // <-- override
4 #include "../.../output_writer.h"
```

Ubicación del programa	Profundidad	OUTPUT_DIR
1erDep_outs/HolaMundo/	1	"../"
2doDep_outs/SortBubble/BubbleArr/	2	"../.../"
2doDep_outs/Pilas/Arreglos/DatoBase/	3	"../...../" (por defecto)

Así, sin importar la profundidad del subdirectorio, los cuatro archivos (`out.json`, `out.xml`, `out.csv`, `out.txt`) terminan siempre en la **raíz** de `1erDep_outs` o `2doDep_outs`, centralizando la salida de cualquier ejercicio que se ejecute.

2.8 Ejemplo de salida generada

Al ejecutar, por ejemplo, el **Ejercicio8**, los cuatro archivos quedan con el mismo contenido lógico en distinto formato:

out.json

```
[
  {
    "programa": "Ejercicio8",
    "salida": "Numero 1: ... Suma: 25\nPromedio: 5\nMaximo: 7\nMinimo: 3\n"
  }
]
```

out.txt

```
=====
PROGRAMA: Ejercicio8
-----
Numero 1: Numero 2: ... Suma: 25
Promedio: 5
Maximo: 7
Minimo: 3
=====
```

Los archivos **out.csv** y **out.xml** contienen la misma información, con el escape de comillas y de entidades respectivamente. Como todos se abren con **trunc**, cada nueva ejecución sustituye a la anterior.

3 Programa EstructurasCasaMetro

3.1 Visión general

EstructurasCasaMetro es un programa interactivo de consola que modela una **red de transporte** (estaciones de Metro y Metrobús de la Ciudad de México) y permite verla y analizarla con tres estructuras de datos distintas, además de calcular rutas óptimas:

1. Cargar la red desde un archivo **.json** o **.xml**.
2. Visualizarla como **grafo no dirigido**.
3. Visualizarla como **dígrafo** (dirigido).
4. Visualizarla como **árbol** (recorrido BFS desde una raíz elegida).
5. Calcular la **ruta más corta** (en minutos) entre dos estaciones con el algoritmo de **Dijkstra**.

La red de ejemplo (**casametro.json**) tiene 5 estaciones y 4 conexiones dirigidas con su *costo* (\$) y *tiempo* (minutos):

```
Metro Aquiles Serdán --(10 min)--> Metro Tacuba --(40 min)--> Metro Xola
Metro Xola --(7 min)--> MB Xola --(18 min)--> MB Iztacalco
```

3.2 Organización de archivos

El proyecto separa interfaz (**include/**) e implementación (**src/**), con un **main.cpp** que orquesta todo.

Archivo	Líneas	Rol
main.cpp	139	Menú, lectura de opciones y orquestación.
include/grafoadt.h	53	Estructuras (Nodo, Arista, Ruta) y la clase base abstracta GrafoADT.
src/grafoadt.cpp	66	cargar, agregarNodo y Dijkstra (común a todas las variantes).
include/grafos.h / src/grafos.cpp	11 / 29	Grafo no dirigido.
include/digrafo.h / src/digrafo.cpp	12 / 27	Dígrafo (dirigido).
include/arbol.h / src/arbol.cpp	26 / 96	Árbol por BFS desde una raíz.
include/parser.h / src/parser.cpp	18 / 222	Lectores de .json y .xml hechos a mano.

Nota sobre la compilación. main.cpp incluye los .cpp directamente (#include "src/grafoadt.cpp", etc.). Esto permite compilar todo con una sola orden: `g++ -std=c++17 -o executable main.cpp`.

3.3 Estructuras de datos

Definidas en `grafoadt.h`, son la base que comparten todas las variantes:

```

1 struct Nodo {                                // una estación
2     int id;
3     std::string etiqueta;                    // nombre legible
4 };
5
6 struct Arista {                               // una conexión origen -> destino
7     std::string origen;
8     std::string destino;
9     std::string costo;                       // p. ej. "$5" (informativo)
10    int tiempo;                             // peso en minutos (usado por Dijkstra)
11 };
12
13 struct Ruta {                                 // resultado de Dijkstra
14     std::vector<std::string> nodos; // secuencia de estaciones
15     int tiempo_total = 0;
16     bool existe = false;
17 };
    
```

3.4 La clase base GrafoADT

GrafoADT es una **clase abstracta** (TAD) que centraliza todo lo común y deja como *virtual puro* sólo lo que cambia entre un grafo, un dígrafo y un árbol: cómo se agrega una arista y cómo se muestra.

```

1 class GrafoADT {
2 protected:
3     std::string nombre_;
4     std::vector<Nodo> nodos_;
5     std::vector<Arista> aristas_;
6     // lista de adyacencia: etiqueta -> [(vecino, peso), ...]
7     std::map<std::string, std::vector<std::pair<std::string,int>>> adj_;
8 public:
9     virtual void agregarNodo(const Nodo& n);
10    virtual void agregarArista(const Arista& a) = 0; // <- cada hijo decide
11    virtual void mostrar() const = 0;               // <- cada hijo decide
    
```



```

12
13     Ruta dijkstra(const std::string& o, const std::string& d) const;
14     void cargar(const std::vector<Nodo>&, const std::vector<Arista>&,
15                 const std::string& nombre);
16     // getters...
17 };
    
```

El corazón es la **lista de adyacencia** `adj_`: un map de cada estación a sus vecinos con el peso. La clave del diseño es que las tres subclases construyen `adj_` de forma distinta (bidireccional, dirigida, ...), pero Dijkstra —que opera sobre `adj_`— se escribe una sola vez en la base.

3.4.1 cargar y agregarNodo

`cargar` limpia el estado, guarda el nombre y vuelca nodos y aristas llamando a `agregarNodo` (común) y `agregarArista` (polimórfico):

```

1 void GrafoADT::agregarNodo(const Nodo& n) {
2     nodos_.push_back(n);
3     adj_[n.etiqueta];           // crea lista vacía si no existe
4 }
5 void GrafoADT::cargar(const std::vector<Nodo>& nodos,
6                       const std::vector<Arista>& aristas,
7                       const std::string& nombre) {
8     nodos_.clear(); aristas_.clear(); adj_.clear();
9     nombre_ = nombre;
10    for (const auto& n : nodos) agregarNodo(n);
11    for (const auto& a : aristas) agregarArista(a); // polimórfico
12 }
    
```

3.4.2 El algoritmo de Dijkstra

Implementa Dijkstra clásico con cola de prioridad (mín-heap). Calcula distancias mínimas en minutos desde el origen y reconstruye la ruta hacia atrás con el mapa `prev`:

```

1 Ruta GrafoADT::dijkstra(const std::string& origen,
2                          const std::string& destino) const {
3     const int INF = std::numeric_limits<int>::max();
4     if (!adj_.count(origen) || !adj_.count(destino)) return {{},0,false};
5
6     std::map<std::string,int> dist;
7     std::map<std::string,std::string> prev;
8     for (const auto& [k,_] : adj_) dist[k] = INF;
9     dist[origen] = 0;
10
11    using Pi = std::pair<int,std::string>;
12    std::priority_queue<Pi,std::vector<Pi>,std::greater<Pi>> pq;
13    pq.push({0, origen});
14
15    while (!pq.empty()) {
16        auto [d,u] = pq.top(); pq.pop();
17        if (d > dist[u]) continue;           // entrada obsoleta
18        for (const auto& [v,w] : adj_.at(u)) {
19            int alt = dist[u] + w;
20            if (alt < dist[v]) {               // relajación
21                dist[v] = alt; prev[v] = u;
22                pq.push({alt, v});
23            }
24        }
25    }
26    if (dist[destino] == INF) return {{},0,false}; // inalcanzable
27    // reconstrucción de la ruta destino -> origen y se invierte...
    
```

28 }

Si no existe camino, devuelve Ruta con `existe = false`.

3.5 Las tres variantes

3.5.1 Grafo (no dirigido)

Cada arista se inserta en `adj_` en **ambos sentidos**, de modo que la conexión vale en las dos direcciones:

```
1 void Grafo::agregarArista(const Arista& a) {
2     aristas_.push_back(a);
3     adj_[a.origen].push_back({a.destino, a.tiempo});
4     adj_[a.destino].push_back({a.origen, a.tiempo}); // bidireccional
5 }
```

3.5.2 Digrafo (dirigido)

La arista se inserta en **un solo sentido** (origen $\rightarrow$ destino). Es la estructura que respeta la dirección real de los viajes y la que usa el menú para Dijkstra.

```
1 void Digrafo::agregarArista(const Arista& a) {
2     aristas_.push_back(a);
3     adj_[a.origen].push_back({a.destino, a.tiempo}); // un solo sentido
4 }
```

3.5.3 Arbol (BFS desde una raíz)

También guarda las aristas dirigidas, pero además ofrece `construirDesdeRaiz`, que hace un **recorrido en anchura (BFS)** desde la estación elegida, registrando para cada nodo sus *hijos* y el conjunto de nodos `alcanzados_`. Los nodos que el BFS no alcanza se listan aparte.

```
1 void Arbol::construirDesdeRaiz(const std::string& raiz) {
2     raiz_ = raiz; hijos_.clear(); alcanzados_.clear();
3     std::queue<std::string> cola;
4     cola.push(raiz_); alcanzados_.insert(raiz_); hijos_[raiz_] = {};
5     while (!cola.empty()) {
6         std::string u = cola.front(); cola.pop();
7         if (!adj_.count(u)) continue;
8         for (const auto& [v,w] : adj_.at(u)) {
9             if (!alcanzados_.count(v)) { // evita ciclos
10                alcanzados_.insert(v);
11                hijos_[u].push_back(v);
12                cola.push(v);
13            }
14        }
15    }
16    // nodos no alcanzables se registran para mostrarlos por separado...
17 }
```

El método `mostrar` (con el auxiliar recursivo `mostrarNodo`) imprime el árbol con conectores $\text{—} / \text{—}$ y el peso de cada arista entre paréntesis.

3.6 El lector de archivos (parser)

`parser.cpp` implementa **lectores de JSON y XML escritos a mano** (sin librerías externas). El punto de entrada `parsearArchivo` detecta la extensión y delega:

```

1 DatosGrafo parsearArchivo(const std::string& nombreArchivo) {
2     // ...obtiene la extensión en minúsculas...
3     if (ext == "json") return parsearJSON(nombreArchivo);
4     if (ext == "xml") return parsearXML(nombreArchivo);
5     throw std::runtime_error("Extension no soportada: ." + ext);
6 }

```

Ambos lectores devuelven la misma estructura `DatosGrafo` (`nombre`, `nodos`, `aristas`). Internamente se apoyan en funciones auxiliares:

- **JSON:** `jsonStr` / `jsonInt` extraen un valor por clave; `jsonSeccion` aísla un arreglo `"clave": [...]` contando corchetes; `nextObj` recorre objeto por objeto contando llaves.
- **XML:** `xmlTags` recolecta todas las etiquetas de un nombre dado y `xmlAttr` lee el valor de un atributo `attr="..."`.
- **Comunes:** `leerArchivo` carga el archivo completo a una cadena y `toMinutos` convierte `"10 min"` en el entero `10`.

Si el campo `id` no aparece, el lector asigna uno automático incremental; una arista sin `origen` o `destino` se descarta. El `costo` se guarda como texto (informativo) y sólo el `tiempo` se convierte a entero para los cálculos.

3.7 El flujo principal (`main.cpp`)

`main` sigue un flujo simple y robusto:

1. Pide el nombre del archivo en un bucle hasta que `parsearArchivo` tenga éxito (captura excepciones y reintenta).
2. Muestra un menú y lee la opción con `leerOpcion`, que valida el rango y limpia la entrada inválida.
3. Según la opción, crea la estructura adecuada (`Grafo`, `Digrafo` o `Arbol`), la carga con los mismos datos y la muestra. Para Dijkstra usa siempre el `Digrafo` (respeta la dirección de las aristas).
4. Repite hasta que el usuario elige `0` (Salir).

```

1 case 4: { // Ruta más corta
2     // ...el usuario elige origen y destino por número...
3     Digrafo dg;
4     dg.cargar(datos.nodos, datos.aristas, datos.nombre);
5     const Ruta ruta = dg.dijkstra(origen, destino);
6     if (!ruta.existe) std::cout << " No existe ruta...\n";
7     else { /* imprime tiempo_total y la secuencia de nodos */ }
8     break;
9 }

```

3.8 Ejemplos de ejecución

A continuación, salidas reales del programa con la red de ejemplo `casametro.json`.

Opción 1 — Grafo no dirigido (la conexión vale en ambos sentidos):

```

GRAFO (No Dirigido): Metro Aquiles Serdán
Nodos : 5
Aristas: 4 (cada una vale en ambas direcciones)

```

Lista de adyacencia:

```
└─ Metro Aquiles Serdán
  │ ↔ Metro Tacuba [10 min]
  │
  └─ Metro Tacuba
    │ ↔ Metro Aquiles Serdán [10 min]
    │ ↔ Metro Xola [40 min]
    │
    └─ Metro Xola
      │ ↔ Metro Tacuba [40 min]
      │ ↔ MB Xola [7 min]
```

Opción 3 — Árbol BFS (raíz: *Metro Aquiles Serdán*):

```
ÁRBOL (BFS desde raíz)
Raíz: Metro Aquiles Serdán
Nodos alcanzables: 5 / 5

[Metro Aquiles Serdán]
└─ (10 min) [Metro Tacuba]
  └─ (40 min) [Metro Xola]
    └─ (7 min) [MB Xola]
      └─ (18 min) [MB Iztacalco]
```

Opción 4 — Dijkstra (de *Metro Aquiles Serdán* a *MB Iztacalco*):

```
Origen : Metro Aquiles Serdán
Destino: MB Iztacalco
Tiempo total: 75 min
Ruta: Metro Aquiles Serdán → Metro Tacuba → Metro Xola → MB Xola → MB Iztacalco
```

3.9 Compilación y ejecución

```
$ cd EstructurasCasaMetro
$ g++ -std=c++17 -o executable main.cpp
$ ./executable
# luego escribir: casametro.json
```

4 Conclusiones

La librería `output_writer.h` resuelve, con un patrón RAII muy compacto, la centralización de salidas: cualquier programa de los dos primeros departamentales puede volcar lo que imprime a cuatro formatos (`.json`, `.xml`, `.csv`, `.txt`) añadiendo apenas tres líneas, y la macro `OUTPUT_DIR` adapta la ruta a la profundidad de cada subdirectorio.

`EstructurasCasaMetro`, por su parte, es un buen ejemplo de **polimorfismo** bien aprovechado: una sola clase base (`GrafoADT`) concentra la lista de adyacencia y el algoritmo de Dijkstra, mientras que las subclases `Grafo`, `Digrafo` y `Arbol` sólo redefinen cómo se construye la adyacencia y cómo se muestra. El lector de archivos hecho a mano demuestra el manejo de JSON y XML sin dependencias externas.